**TRƯỜNG ĐẠI HỌC LẠC HỒNG**
**KHOA CÔNG NGHỆ THÔNG TIN**

---

# BÁO CÁO ĐỒ ÁN CUỐI KỲ
# HỌC PHẦN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

# ĐỀ TÀI: TÌM ĐƯỜNG ĐI NGẮN NHẤT TRÊN BẢN ĐỒ BẰNG THUẬT TOÁN DIJKSTRA VÀ A*

*Cài đặt thuần cấu trúc Min-Heap có mảng chỉ mục, Bảng băm DJB2 và thực nghiệm đa quy mô*

Giảng viên hướng dẫn: ThS. Phan Mạnh Thường
Sinh viên thực hiện:
1. Nguyễn Hoàng Anh - 25CT111 - 125000977
2. Hà Văn Kiên - 25CT111 - 125000399
3. Trần Quang Huy - 25CT111 - 125001825
4. Hoàng Minh Phát - 25CT111 - 125001323

*Đồng Nai, ngày 11 tháng 11 năm 2026*

# MỤC LỤC

LỜI MỞ ĐẦU ..... 3
CHƯƠNG 1. BÀI TOÁN VÀ MÔ HÌNH HÓA HỆ THỐNG ..... 3
1.1. Bài toán tìm đường đi ngắn nhất trong giao thông đô thị ..... 3
1.2. Chuẩn đầu ra môn học và yêu cầu đề tài ..... 4
1.3. Kế hoạch làm việc nhóm và phân công nhiệm vụ chi tiết ..... 4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ LỰA CHỌN GIẢI PHÁP (CLO3) ..... 5
2.1. Đồ thị và cách biểu diễn ..... 5
2.2. Hàng đợi ưu tiên và Min-Heap ..... 5
2.3. Thuật toán Dijkstra ..... 6
2.4. Thuật toán A* và tính Admissible của Heuristic ..... 7
2.5. Các cấu trúc phụ trợ và thuật toán số học ..... 7
2.6. So sánh tổng hợp ưu và nhược điểm các phương án giải pháp (CLO3) ..... 8
CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG (CLO2) ..... 9
3.1. Kiến trúc hệ thống tổng thể ..... 9
3.2. Các lớp cấu trúc dữ liệu nền tảng ..... 10
3.3. Cấu trúc đồ thị CityMapGraph ..... 11
3.4. Cài đặt thuật toán Dijkstra và A* ..... 11
3.5. Xử lý các trường hợp biên và điều kiện bất thường ..... 13
3.6. Cài đặt và phát triển trên nền tảng C# và Python (CLO2) ..... 14
CHƯƠNG 4. MINH HỌA THUẬT TOÁN TRÊN BẢN ĐỒ MẪU ..... 14
4.1. Bảng vết chạy từng bước của thuật toán Dijkstra ..... 14
4.2. Bảng vết chạy từng bước của thuật toán A* ..... 15
4.3. Mô phỏng kẹt xe động và tự động chuyển hướng ..... 16
CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ THỰC NGHIỆM (CLO4) ..... 17
5.1. Phương pháp và môi trường thực nghiệm ..... 17
5.2. Kết quả kiểm thử đúng đắn tự động ..... 17
5.3. Đánh giá hiệu năng và bộ nhớ sử dụng đa quy mô (CLO4) ..... 19
5.4. Hạn chế và hướng phát triển ..... 20
TÀI LIỆU THAM KHẢO ..... 21

# LỜI MỞ ĐẦU

Trong kỷ nguyên số hóa đô thị và giao thông thông minh (Intelligent Transportation Systems - ITS), việc tối ưu hóa hành trình di chuyển giữ vai trò huyết mạch trong việc giảm thiểu khí thải ô nhiễm môi trường, tiết kiệm thời gian và nhiên liệu xã hội, đồng thời giảm tải áp lực ùn tắc lên hạ tầng giao thông đô thị. Bài toán tìm đường đi ngắn nhất trên mạng lưới đường bộ thực tế là bài toán kinh điển của Khoa học máy tính, đòi hỏi sự kết hợp hài hòa, chặt chẽ giữa các cấu trúc dữ liệu tiên tiến và các chiến lược giải thuật tìm kiếm tối ưu hóa.

Mục tiêu trọng tâm của đồ án là thiết kế, cài đặt thuần túy (Zero Libraries) và đánh giá thực nghiệm toàn diện hệ thống tìm đường đi ngắn nhất ứng dụng hai thuật toán nền tảng Dijkstra và A* trên đồ thị có hướng mang trọng số không âm. Toàn bộ hệ thống được xây dựng với nguyên tắc tự cài đặt thủ công 100% các cấu trúc dữ liệu lõi bao gồm Hàng đợi ưu tiên Min-Heap có mảng chỉ mục pos[], Bảng băm đóng dò tuyến tính giải quyết xung đột, Ngăn xếp đảo ngược lộ trình và Hàng đợi kiểm tra liên thông BFS. Cách tiếp cận này giúp nhóm sinh viên nắm bắt tường tận bản chất cấu trúc ô nhớ, kỹ thuật quản lý con trỏ, phép toán băm và cơ chế giảm khóa đạt độ phức tạp tiệm cận lý thuyết tối ưu.

Phạm vi nghiên cứu bao gồm việc mô hình hóa mạng lưới giao thông đường bộ dạng đồ thị danh sách kề với tọa độ địa lý chuẩn hóa ki-lô-mét, hỗ trợ các tuyến đường một chiều, hai chiều và mô phỏng hệ số kẹt xe động theo thời gian thực. Bên cạnh đó, đồ án thực hiện đánh giá thực nghiệm toàn diện trên các tập dữ liệu từ bản đồ mẫu đô thị 15 nút đến các đồ thị lưới quy mô lớn đạt ngưỡng 50.000 phần tử nhằm đối chiếu trực tiếp giữa lý thuyết độ phức tạp tiệm cận và thời gian thực thi, bộ nhớ đo đạc trên hệ thống máy tính thực tế.

Bố cục báo cáo được tổ chức thành năm chương chính: Chương 1 trình bày bài toán, mô hình hóa hệ thống và kế hoạch làm việc nhóm; Chương 2 phân tích sâu cơ sở lý thuyết cấu trúc dữ liệu và giải thuật (CLO3); Chương 3 mô tả kiến trúc thiết kế, giải pháp cài đặt chi tiết trên C# và Python (CLO2); Chương 4 minh họa từng bước chạy thuật toán trên bản đồ thực tế; và Chương 5 tổng kết toàn bộ kiểm thử đúng đắn cùng thực nghiệm benchmark hiệu năng chuyên sâu đến 50.000 phần tử (CLO4).

# CHƯƠNG 1. BÀI TOÁN VÀ MÔ HÌNH HÓA HỆ THỐNG

## 1.1. Bài toán tìm đường đi ngắn nhất trong giao thông đô thị

Bài toán đường đi ngắn nhất trên đồ thị có hướng là bài toán tìm kiếm một chuỗi liên tiếp các cung nối từ đỉnh xuất phát s đến đỉnh đích t sao cho tổng trọng số hiệu dụng tích lũy trên các cung là nhỏ nhất. Trong ngữ cảnh bản đồ giao thông đường bộ, một mạng lưới đường sá được mô hình hóa tự nhiên bằng đồ thị có hướng G = (V, E), trong đó tập đỉnh V biểu diễn tập hợp các giao lộ, bến xe, công viên hoặc các địa điểm trọng yếu, còn tập cung E đại diện cho các tuyến đường kết nối trực tiếp giữa các địa điểm đó. Mỗi cung có hướng e = (u, v) được gán một trọng số cơ sở w(u, v) biểu thị khoảng cách vật lý thực tế được đo lường bằng đơn vị ki-lô-mét.

Để phản ánh chính xác quy hoạch giao thông đô thị hiện đại, hệ thống phân biệt tường minh giữa tuyến đường một chiều và hai chiều. Tuyến đường một chiều từ đỉnh u đến đỉnh v chỉ cho phép phương tiện lưu thông theo chiều thuận e = (u, v), do đó chỉ cung này xuất hiện trong danh sách liên kết kề của đỉnh u mà không có chiều ngược lại. Ngược lại, tuyến đường hai chiều giữa hai đỉnh được mô hình hóa đối xứng bằng hai cung riêng biệt (u, v) và (v, u) có cùng độ dài vật lý cơ sở, cho phép các thuật toán điều hướng dễ dàng xử lý tương thích mà không làm mất tính tổng quát.

Yếu tố biến động giao thông thực tế được trừu tượng hóa thông qua hệ số kẹt xe $k \geq 1$ được gán động cho từng tuyến đường. Khi xảy ra tình trạng ùn tắc tại một đoạn đường cụ thể, trọng số hiệu dụng dùng cho việc tính toán điều hướng được xác định theo nguyên lý tỷ lệ thuận bằng tích số giữa chiều dài cơ sở và hệ số kẹt xe tương ứng:

$$w_{\text{eff}}(u, v) = w(u, v) \times k(u, v), k(u, v) \geq 1.0 \quad (1)$$

Hệ số kẹt xe $k = 1.0$ tương ứng với trạng thái đường thông thoáng lý tưởng khi vận tốc phương tiện đạt mức thiết kế tối đa, trong khi $k > 1.0$ mô phỏng tình trạng lưu lượng xe tăng cao làm giảm tốc độ hành trình, tương đương với việc gia tăng chi phí thời gian di chuyển trên đoạn đường đó. Giới hạn $k \geq 1.0$ đảm bảo tính nhất quán vật lý vì không có hiện tượng giao thông tự nhiên nào làm thời gian di chuyển giảm xuống dưới ngưỡng vận tốc thiết kế tối đa của cung đường.

Trong mô hình toán học của đề tài, các cạnh mang trọng số âm hoàn toàn bị loại bỏ bởi hai lý do cơ bản. Về mặt vật lý giao thông, quãng đường di chuyển và thời gian tiêu tốn giữa hai địa điểm luôn là các đại lượng thực dương, không thể tồn tại một cung đường mà việc đi qua nó lại làm giảm quãng đường tích lũy trước đó của phương tiện. Về mặt lý thuyết tính toán, sự xuất hiện của cạnh âm sẽ phá vỡ bất biến tham lam nền tảng của thuật toán Dijkstra, đồng thời nếu tồn tại chu trình có tổng trọng số âm thì đường đi ngắn nhất sẽ trở nên không xác định vì phương tiện có thể lặp vô hạn quanh chu trình để chi phí giảm về âm vô cực.

## 1.2. Chuẩn đầu ra môn học và yêu cầu đề tài

Bám sát Đề cương chi tiết học phần Cấu trúc Dữ liệu và Giải thuật, toàn bộ đồ án Đề tài 5 được thiết kế nhằm hiện thực hóa và đáp ứng xuất sắc bốn chuẩn đầu ra cốt lõi (Course Learning Outcomes - CLO):

1. Chuẩn đầu ra CLO3 (Nghiên cứu và Lựa chọn): Phân tích, so sánh các cấu trúc dữ liệu đồ thị (Ma trận kề vs Danh sách kề), cấu trúc hàng đợi ưu tiên (Mảng vs Min-Heap vs Indexed Min-Heap) và các giải thuật tìm đường (Dijkstra vs Bellman-Ford vs Floyd-Warshall vs A*), đưa ra lập luận thuyết phục về việc lựa chọn giải pháp tối ưu tiệm cận thời gian $\mathcal{O}((V+E)\log V)$ và không gian $\mathcal{O}(V+E)$.

2. Chuẩn đầu ra CLO2 (Cài đặt và Phát triển): Sử dụng ngôn ngữ lập trình C# (.NET 9) kết hợp ngôn ngữ Python để cài đặt thuần túy 100% toàn bộ cấu trúc dữ liệu và giải thuật không phụ thuộc thư viện ngoài; xây dựng ứng dụng hoàn chỉnh có cả giao diện dòng lệnh (CLI) và giao diện trực quan mô phỏng (Web GUI/Canvas); mã nguồn tuân thủ nghiêm ngặt các chuẩn lập trình hướng đối tượng.

3. Chuẩn đầu ra CLO4 (Thực nghiệm và Đánh giá): Xây dựng bộ kiểm thử đa dạng gồm 26 ca kiểm thử tự động, kiểm thử tính đúng đắn trên 200 cặp đỉnh ngẫu nhiên, thực hiện đo đạc cả thời gian thực thi (ms) và bộ nhớ tiêu thụ (MB) trên các quy mô đồ thị từ nhỏ (15 nút) đến lớn (100, 1.000, 5.000, 20.000 và 50.000 phần tử), trình bày trực quan bằng bảng và biểu đồ so sánh.

4. Chuẩn đầu ra CLO5 (Báo cáo và Kỹ năng): Hoàn thiện bản báo cáo khoa học chi tiết, lập kế hoạch làm việc nhóm bài bản có phân công nhiệm vụ cụ thể cho từng thành viên, xây dựng biểu đồ tiến độ Gantt và chuẩn bị bài thuyết trình slide bảo vệ trước hội đồng.

## 1.3. Kế hoạch làm việc nhóm và phân công nhiệm vụ chi tiết

Để đảm bảo đồ án hoàn thành đúng tiến độ và đạt chất lượng học thuật cao nhất, nhóm 4 thành viên đã xây dựng kế hoạch hành động phân rã công việc (Work Breakdown Structure - WBS) và thiết lập ma trận trách nhiệm RACI rõ ràng. Bảng 1.1 tổng hợp vai trò, nhiệm vụ phụ trách chính và tỷ lệ đóng góp của từng thành viên trong nhóm:

*Bảng 1.1. Bảng phân công nhiệm vụ chi tiết và mức độ đóng góp của từng thành viên nhóm*

| STT | Họ và tên | MSSV | Vai trò trong nhóm | Nhiệm vụ phụ trách chính | Đóng góp |
|---|---|---|---|---|---|
| 1 | Nguyễn Hoàng Anh | 125000977 | Nhóm trưởng (Lead Dev) | Thiết kế kiến trúc hệ thống; Cài đặt cấu trúc dữ liệu IndexedMinHeap, CustomHashTable; Cài đặt thuật toán Dijkstra & A*; Chủ biên báo cáo Word. | 100% |
| 2 | Hà Văn Kiên | 125000399 | Thành viên (Core Dev) | Thiết kế mô hình đồ thị CityMapGraph, xử lý cung một chiều và kẹt xe động; Cài đặt phiên bản C# (.NET 9); Soạn slide thuyết trình PPTX. | 100% |
| 3 | Trần Quang Huy | 125001825 | Thành viên (Data & Bench) | Xây dựng bộ sinh đồ thị lưới và ngẫu nhiên đa quy mô (đến 50.000 phần tử); Đo đạc thời gian, bộ nhớ; Vẽ biểu đồ phân tích thực nghiệm. | 100% |

| STT | Họ và tên | MSSV | Vai trò trong nhóm | Nhiệm vụ phụ trách chính | Đóng góp |
|---|---|---|---|---|---|
| 4 | Hoàng Minh Phát | 125001323 | Thành viên (QA & Frontend) | Xây dựng bộ kiểm thử tự động (26 Test Cases); Phát triển giao diện Web Canvas mô phỏng trực quan; Quay và dựng video demo hệ thống. | 100% |

Dự án được triển khai xuyên suốt qua 6 tuần làm việc tập trung, bám sát các mốc kiểm soát tiến độ định kỳ. Bảng 1.2 mô tả chi tiết các giai đoạn thực hiện dự án:

*Bảng 1.2. Kế hoạch tiến độ thực hiện đồ án qua 6 tuần làm việc*

| Giai đoạn | Thời gian | Nội dung công việc chủ yếu | Sản phẩm bàn giao | Người phụ trách |
|---|---|---|---|---|
| Tuần 1 | 01/10 - 07/10 | Khảo sát đề bài, nghiên cứu tài liệu lý thuyết đồ thị và giải thuật tìm đường. | Đề cương chi tiết và tài liệu khảo sát | Cả nhóm |
| Tuần 2 | 08/10 - 14/10 | Thiết kế cấu trúc dữ liệu: Min-Heap, Hash Table, Stack, Queue không dùng thư viện. | Mã nguồn các lớp CTDL nền tảng | Hoàng Anh, Hà Kiên |
| Tuần 3 | 15/10 - 21/10 | Cài đặt thuật toán Dijkstra và A* Admissible; Xây dựng bản đồ mẫu 15 nút Bến Tre. | Mô-đun tìm đường cốt lõi | Hoàng Anh, Hà Kiên |
| Tuần 4 | 22/10 - 28/10 | Xây dựng bộ kiểm thử 26 ca test; Viết mã C# song song; Phát triển giao diện Web Canvas. | Bộ test tự động và ứng dụng Web | Quang Huy, Minh Phát |
| Tuần 5 | 29/10 - 04/11 | Thực hiện stress-test đa quy mô đến 50.000 phần tử; Đo đạc thời gian, bộ nhớ; Vẽ biểu đồ. | Báo cáo thực nghiệm và biểu đồ đo | Quang Huy, Minh Phát |
| Tuần 6 | 05/11 - 10/11 | Tổng hợp báo cáo thuyết minh Word, hoàn thiện slide PowerPoint và quay video demo. | Báo cáo Word, Slide PPTX, Video | Cả nhóm |

# CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ LỰA CHỌN GIẢI PHÁP (CLO3)

## 2.1. Đồ thị và cách biểu diễn

Trong lý thuyết đồ thị và khoa học máy tính, hai phương pháp biểu diễn đồ thị phổ biến nhất là ma trận kề (adjacency matrix) và danh sách kề (adjacency list). Ma trận kề sử dụng một mảng hai chiều kích thước $V \times V$, trong đó phần tử tại vị trí hàng u và cột v lưu trữ trọng số của cung nối từ u đến v, hoặc nhận giá trị vô cùng nếu không tồn tại cạnh nối trực tiếp. Phương pháp này có ưu điểm cho phép kiểm tra sự tồn tại của một cung bất kỳ trong thời gian hằng số $\mathcal{O}(1)$, song nhược điểm chí mạng là đòi hỏi không gian bộ nhớ cố định $\mathcal{O}(V^2)$, bất kể số lượng cạnh thực tế nhiều hay ít.

Ngược lại, danh sách kề biểu diễn đồ thị bằng một mảng gồm V phần tử, trong đó mỗi phần tử chứa con trỏ trỏ đến đầu một danh sách liên kết đơn lưu trữ tập hợp các đỉnh kề trực tiếp cùng thuộc tính trọng số tương ứng. Tổng bộ nhớ tiêu thụ của danh sách kề chỉ tỷ lệ thuận với số lượng đỉnh và số lượng cung thực tế là $\mathcal{O}(V + E)$. Trong các mạng lưới giao thông đường bộ đô thị, đồ thị luôn mang bản chất thưa rệt khi mỗi nút giao thông thông thường chỉ kết nối từ 2 đến 4 tuyến đường rẽ, dẫn đến số cạnh xấp xỉ tuyến tính với số đỉnh $E \approx k \cdot V$ với $k \leq 4$. Với quy mô 20.000 đến 50.000 đỉnh, ma trận kề đòi hỏi bảng từ 400 triệu đến 2.5 tỷ ô nhớ gây tràn bộ nhớ RAM, trong khi danh sách kề chỉ yêu cầu khoảng 100.000 đến 250.000 phần tử liên kết, do đó danh sách kề là sự lựa chọn tối ưu tuyệt đối cho đồ án.

## 2.2. Hàng đợi ưu tiên và Min-Heap

Hàng đợi ưu tiên (priority queue) là cấu trúc dữ liệu trừu tượng quản lý tập hợp các phần tử có gắn kèm khóa độ ưu tiên, hỗ trợ trích xuất phần tử có khóa nhỏ nhất một cách hiệu quả. Min-Heap là một hiện thực kinh điển của hàng đợi ưu tiên dưới dạng cây nhị phân gần đầy đủ (nearly complete binary tree), thỏa mãn tính chất đống: khóa của mỗi nút cha luôn nhỏ hơn hoặc bằng khóa của hai nút con

tương ứng. Cấu trúc này được lưu trữ liên tục trên một mảng một chiều tuyến tính mà không cần cấp phát con trỏ động, giúp tiết kiệm bộ nhớ và tối ưu hóa bộ nhớ đệm CPU.

Với quy ước chỉ số mảng bắt đầu từ 0, mối quan hệ phân cấp giữa chỉ số nút cha và các nút con được xác định hoàn toàn thông qua các công thức số học nguyên:

$$\text{parent}(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \tag{2}$$

$$\text{left_child}(i) = 2i + 1 \tag{3}$$

$$\text{right_child}(i) = 2i + 2 \tag{4}$$

Thao tác chèn một phần tử mới insert(priority, item) được thực hiện bằng cách bổ sung phần tử vào cuối mảng rồi kích hoạt thủ tục vun lên sift-up. Thao tác trích xuất đỉnh nhỏ nhất pop_min() lấy phần tử gốc tại chỉ số 0, đưa phần tử cuối cùng lên thay thế gốc và kích hoạt thủ tục vun xuống sift-down để tái lập bất biến đống. Vì cây nhị phân gần đầy đủ chứa n phần tử có chiều cao h được chặn trên bởi $h = \lfloor \log_2(n) \rfloor$, số lượng phép tráo đổi nút trong quá trình vun lên hoặc vun xuống dọc theo một đường đi từ gốc tới lá không bao giờ vượt quá chiều cao cây, chứng minh độ phức tạp thời gian của cả hai thao tác đều đạt $\mathcal{O}(\log n)$.

Trong thuật toán tìm đường đi ngắn nhất, khi một tuyến đường tốt hơn tới đỉnh v được phát hiện, ta cần cập nhật giảm khoảng cách của đỉnh v trong hàng đợi ưu tiên, thao tác này được gọi là decrease-key. Nếu sử dụng heap tiêu chuẩn, việc định vị đỉnh v trong heap đòi hỏi quét tuyến tính toàn bộ mảng mất thời gian $\mathcal{O}(n)$, làm suy thoái độ phức tạp của thuật toán Dijkstra lên $\mathcal{O}(V^2)$. Để giải quyết triệt để vấn đề này, đề tài cài đặt IndexedMinHeap tích hợp một mảng vị trí pos[] kích thước V, trong đó pos[$v$] lưu trữ chính xác chỉ số vị trí hiện tại của đỉnh v trong mảng heap, cho phép tra cứu tọa độ đỉnh trong $\mathcal{O}(1)$ và thực hiện vun lên sift-up trong $\mathcal{O}(\log V)$. Cách tiếp cận này vượt trội hoàn toàn so với kỹ thuật chèn trùng nút (lazy deletion) vốn làm kích thước heap phình to lên $\mathcal{O}(E)$ phần tử và tiêu tốn thêm thời gian xử lý các bản ghi rác.

## 2.3. Thuật toán Dijkstra

Thuật toán Dijkstra do nhà khoa học máy tính Edsger W. Dijkstra đề xuất năm 1959 là thuật toán tham lam kinh điển giải quyết bài toán đường đi ngắn nhất từ một nguồn đơn trên đồ thị có trọng số không âm. Thuật toán duy trì một mảng khoảng cách ước lượng dist[$v$] cho mọi đỉnh v, khởi tạo dist[$s$] = 0 cho đỉnh xuất phát s và dist[$v$] = $\infty$ cho tất cả các đỉnh còn lại. Tại mỗi bước lặp, thuật toán trích xuất đỉnh u có giá trị dist[$u$] nhỏ nhất trong số các đỉnh chưa hoàn tất, chốt đỉnh u là tối ưu, sau đó thực hiện thao tác nới lỏng (relaxation) trên tất cả các cung đi ra (u, v) theo công thức toán học:

$$\text{dist}[v] \leftarrow min(\text{dist}[v],\ \text{dist}[u] + w_{\text{eff}}(u, v)) \tag{5}$$

Tính đúng đắn của thuật toán Dijkstra được chứng minh chặt chẽ bằng phương pháp quy nạp toán học dựa trên bất biến vòng lặp: tại thời điểm một đỉnh u bất kỳ được trích xuất ra khỏi hàng đợi ưu tiên, giá trị dist[$u$] đã đạt tới khoảng cách ngắn nhất tuyệt đối $\delta(s, u)$. Cơ sở quy nạp đúng tại bước khởi tạo khi đỉnh s được lấy ra đầu tiên với dist[$s$] = 0. Giả sử bất biến đúng cho tập hợp các đỉnh đã chốt S. Khi chọn đỉnh kế tiếp u không thuộc S có dist[$u$] nhỏ nhất, giả sử phản chứng tồn tại một đường đi thực tế P nối s tới u có tổng chiều dài ngắn hơn dist[$u$]. Khi đó đường đi P phải rời tập S tại một cung nào đó (x, y) với x thuộc S và y chưa thuộc S. Vì x đã được chốt tối ưu và đã nới lỏng cạnh (x, y), ta có dist[$y$] $\leq$ length($P_{s \to y}$). Do trọng số mọi cung đều không âm, chiều dài từ s đến y không vượt quá chiều dài toàn bộ đường đi P, dẫn tới dist[$y$] $\leq$ length($P$) $<$ dist[$u$]. Tuy nhiên điều này mâu thuẫn trực tiếp với quy tắc chọn đỉnh tham lam rằng u là đỉnh chưa chốt có dist nhỏ nhất trong hàng đợi ưu tiên, chứng tỏ không thể tồn tại đường đi P nào ngắn hơn dist[$u$].

Về mặt độ phức tạp tính toán, thuật toán thực hiện chính xác V lần thao tác trích xuất đỉnh pop_min() và nhiều nhất E lần thao tác nới lỏng cạnh dẫn đến thao tác giảm khóa decrease_key() trên IndexedMinHeap. Do mỗi thao tác trên heap nhị phân có chi phí thời gian $\mathcal{O}(\log V)$, tổng thời gian thực thi của thuật toán đạt mức chặn tiệm cận $\mathcal{O}((V + E)\log V)$.

Giả thiết trọng số không âm đóng vai trò sinh tử đối với tính đúng đắn của phép chứng minh quy nạp nói trên. Khi xuất hiện cạnh mang trọng số âm, một đường đi vòng qua đỉnh khác có thể tạo ra tổng chi phí nhỏ hơn khoảng cách của đỉnh đã chốt, khiến bất biến bị phá vỡ hoàn toàn. Ví dụ, xét đồ thị ba đỉnh gồm cung (A, B) có trọng số 3, cung (A, C) có trọng số 5 và cung (C, B) có trọng số âm

bằng -4. Thuật toán Dijkstra sẽ tham lam chốt đỉnh B ngay từ bước đầu với khoảng cách dist$[B] = 3$ vì $3 < 5$. Tuy nhiên, đường đi thực tế $A \to C \to B$ lại có tổng chi phí bằng $5 + (-4) = 1$, nhỏ hơn 3, chứng minh tính đúng đắn bị phá hỏng hoàn toàn khi tồn tại trọng số âm.

## 2.4. Thuật toán A* và tính Admissible của Heuristic

Thuật toán A* do Peter Hart, Nils Nilsson và Bertram Raphael công bố năm 1968 là thuật toán tìm kiếm theo thông tin (informed search) mở rộng từ Dijkstra nhằm định hướng không gian tìm kiếm về phía đỉnh mục tiêu t. Thuật toán đánh giá độ ưu tiên của mỗi đỉnh n thông qua hàm mục tiêu tổng hợp $f(n)$ kết hợp giữa chi phí đã tích lũy và chi phí ước lượng tương lai:

$$f(n) = g(n) + h(n) \tag{6}$$

Trong đó $g(n)$ biểu diễn chi phí thực tế nhỏ nhất từ đỉnh xuất phát s đến đỉnh n đã tìm được, còn $h(n)$ là hàm heuristic đánh giá khoảng cách ước lượng từ đỉnh n đến đỉnh đích t. Trên bản đồ tọa độ phẳng, hàm heuristic tự nhiên nhất là khoảng cách đường chim bay Euclid giữa hai điểm tọa độ. Khoảng cách Euclid luôn là cận dưới toán học của quãng đường thực tế vì đường thẳng luôn là khoảng cách ngắn nhất giữa hai điểm trong không gian phẳng.

Một hàm heuristic $h(n)$ được gọi là chấp nhận được (admissible) nếu với mọi đỉnh n, giá trị ước lượng $h(n)$ không bao giờ vượt quá khoảng cách ngắn nhất thực tế $h^*(n)$ từ n đến đỉnh đích t, tức thỏa mãn:

$$0 \le h(n) \le h^*(n), \forall n \in V \tag{7}$$

Hàm heuristic $h(n)$ được gọi là nhất quán (consistent hay monotone) nếu với mọi cung (u, v) trên đồ thị, giá trị ước lượng thỏa mãn bất đẳng thức tam giác:

$$h(u) \le w_{\text{eff}}(u, v) + h(v), h(t) = 0 \tag{8}$$

Khi hàm heuristic thỏa mãn tính nhất quán, giá trị hàm $f(n)$ luôn đơn điệu không giảm dọc theo mọi đường đi hợp lệ. Khi một đỉnh u được trích xuất ra khỏi hàng đợi ưu tiên, giá trị $g(u)$ được đảm bảo đã đạt mức chi phí tối ưu tuyệt đối mà không bao giờ cần phải mở lại (re-open) đỉnh đó, giúp thuật toán A* bảo toàn trọn vẹn kết quả đường đi ngắn nhất hoàn toàn trùng khớp với thuật toán Dijkstra. Bản thân thuật toán Dijkstra thực chất chính là một trường hợp đặc biệt của thuật toán A* khi hàm heuristic $h(n) = 0$ với mọi đỉnh n.

Trong các hệ thống bản đồ cho phép người dùng tự do chỉnh sửa hoặc nhập trọng số cạnh thủ công, có thể xảy ra trường hợp trọng số cơ sở của một cung nhỏ hơn khoảng cách hình học Euclid giữa hai đầu mút của nó. Nếu giữ nguyên khoảng cách Euclid thuần túy làm hàm đánh giá, hàm heuristic sẽ vượt trần khoảng cách thực tế và đánh mất tính admissible, dẫn đến nguy cơ A* đưa ra lộ trình sai lệch. Để triệt tiêu hoàn toàn rủi ro này, chương trình tự động tính toán một hệ số tỷ lệ co giãn an toàn $h_$scale theo công thức cực tiểu hóa trên toàn bộ tập cung của đồ thị:

$$h_\text{scale} = max\left(0,\ min\left(1.0,\ min_{(u,v)\in E,\ d_E>0} \frac{w(u,v)}{d_{\text{Euclid}}(u,v)}\right)\right) \tag{9}$$

Nhờ có hệ số $h_$scale, hàm heuristic $h(n) = h_\text{scale} \times d_{\text{Euclid}}(n, t)$ luôn được bảo đảm là một cận dưới chặt chẽ của quãng đường thực tế trong mọi tình huống.

## 2.5. Các cấu trúc phụ trợ và thuật toán số học

Nhằm đảm bảo tôn chỉ không phụ thuộc vào thư viện ngoài, hệ thống đã tự xây dựng ba cấu trúc và giải thuật số học phụ trợ thiết yếu. Cấu trúc thứ nhất là bảng băm CustomHashTable sử dụng hàm băm DJB2 kết hợp kỹ thuật giải quyết xung đột bằng phương pháp dò tuyến tính (linear probing). Hàm băm DJB2 khởi tạo giá trị băm h = 5381 và liên tục nhân với 33 kết hợp cộng mã ký tự tiếp theo:

$$\text{hash}_i = (\text{hash}_{i-1} \times 33 + c_i)(\text{modCapacity}) \tag{10}$$

Để kiểm soát xung đột khi bảng băm đầy dần, bảng băm tự động giám sát hệ số tải $\alpha = \text{size/capacity}$. Khi $\alpha \ge 0.5$, hàm _rehash() được kích hoạt để mở rộng kích thước mảng lên $\text{capacity}_{\text{mới}} = 2 \times \text{capacity}_{\text{cũ}} + 1$ và băm lại toàn bộ các khóa hiện có. Nhờ kiểm soát hệ số tải dưới ngưỡng 0.5, độ phức tạp trung bình cho các thao tác tra cứu, thêm và kiểm tra tồn tại luôn được duy trì ở mức $O(1)$.

Cấu trúc thứ hai là ngăn xếp CustomStack (LIFO) dùng để đảo chiều mảng dấu vết prev[] nhằm phục hồi đường đi từ điểm xuất phát tới điểm đích trong $\mathcal{O}(V)$, và hàng đợi CustomQueue (FIFO) phục vụ thuật toán duyệt BFS kiểm tra tính liên thông của đồ thị.

Thuật toán số học thứ ba là phép tính căn bậc hai số thực sqrt_newton phục vụ tính khoảng cách Euclid mà không sử dụng thư viện ngoài. Giải thuật áp dụng phương pháp lặp tiếp tuyến Newton-Raphson:

$$x_{n+1} = \frac{1}{2}\left(x_n + \frac{S}{x_n}\right) \tag{11}$$

## 2.6. So sánh tổng hợp ưu và nhược điểm các phương án giải pháp (CLO3)

Nhằm đáp ứng toàn diện chuẩn đầu ra CLO3 về việc nghiên cứu, so sánh và lựa chọn giải pháp tối ưu cho đề tài, dưới đây là các bảng phân tích so sánh chi tiết giữa các phương án cấu trúc dữ liệu và giải thuật khả thi:

*Bảng 2.1. So sánh các cấu trúc dữ liệu biểu diễn đồ thị bản đồ giao thông*

| Phương án biểu diễn | Không gian bộ nhớ | Kiểm tra cạnh (u, v) | Duyệt mọi đỉnh kề | Ưu điểm | Nhược điểm & Đánh giá |
|---|---|---|---|---|---|
| Ma trận kề (Adjacency Matrix) | $\mathcal{O}(V^2)$ | $\mathcal{O}(1)$ | $\mathcal{O}(V)$ | Kiểm tra cạnh cực nhanh $\mathcal{O}(1)$; Dễ cài đặt mảng 2 chiều. | Lãng phí bộ nhớ khủng khiếp khi V lớn (50.000 đỉnh cần 2.5 tỷ ô nhớ); Không phù hợp đồ thị thưa. |
| Danh sách cạnh (Edge List) | $\mathcal{O}(E)$ | $\mathcal{O}(E)$ | $\mathcal{O}(E)$ | Tiết kiệm bộ nhớ; Dễ cài đặt cho thuật toán Kruskal. | Thời gian tìm đỉnh kề rất chậm $\mathcal{O}(E)$, làm thuật toán Dijkstra suy thoái thành $\mathcal{O}(V \cdot E)$. |
| Danh sách kề (Adjacency List) [ĐƯỢC CHỌN] | $\mathcal{O}(V+E)$ | $\mathcal{O}(\deg(u))$ | $\mathcal{O}(\deg(u))$ | Tối ưu bộ nhớ hoàn hảo cho đồ thị thưa; Duyệt đỉnh kề trong $\mathcal{O}(\deg(u))$; Khớp tối ưu với Dijkstra. | Kiểm tra cạnh (u, v) tốn $\mathcal{O}(\deg(u))$ thay vì $\mathcal{O}(1)$, nhưng với bậc đỉnh de$g(u)$ <= 4 thì xấp xỉ hằng số. |

Bảng 2.2 so sánh các kỹ thuật cài đặt Hàng đợi ưu tiên phục vụ thao tác lấy đỉnh nhỏ nhất và giảm khóa:

*Bảng 2.2. So sánh các kỹ thuật cài đặt Hàng đợi ưu tiên (Priority Queue)*

| Cấu trúc cài đặt | Insert | Pop-Min | Decrease-Key | Tổng chi phí Dijkstra | Đánh giá lựa chọn |
|---|---|---|---|---|---|
| Mảng chưa sắp xếp | $\mathcal{O}(1)$ | $\mathcal{O}(V)$ | $\mathcal{O}(1)$ | $\mathcal{O}(V^2+E)$ | Chỉ tốt khi đồ thị cực dày (E xấp xỉ $V^2$); Rất chậm trên đồ thị thưa. |
| Mảng có thứ tự | $\mathcal{O}(V)$ | $\mathcal{O}(1)$ | $\mathcal{O}(V)$ | $\mathcal{O}(V^2+E \cdot V)$ | Chèn và giảm khóa quá chậm $\mathcal{O}(V)$, không khả thi. |
| Binary Heap chuẩn (Không có mảng pos) | $\mathcal{O}(\log V)$ | $\mathcal{O}(\log V)$ | $\mathcal{O}(V)$ (quét mảng) | $\mathcal{O}(V\log V + E \cdot V)$ | Thao tác giảm khóa mất $\mathcal{O}(V)$ vì không biết đỉnh v đang nằm ở đâu trong cây heap. |

| Cấu trúc cài đặt | Insert | Pop-Min | Decrease-Key | Tổng chi phí Dijkstra | Đánh giá lựa chọn |
|---|---|---|---|---|---|
| Indexed Min-Heap (Tích hợp mảng pos[]) [ĐƯỢC CHỌN] | $\mathcal{O}(\log V)$ | $\mathcal{O}(\log V)$ | $\mathcal{O}(\log V)$ | $\mathcal{O}((V+E)\log V)$ | Tối ưu tuyệt đối; Mảng pos[] định vị $\mathcal{O}(1)$, vun lên $\mathcal{O}(\log V)$; Đạt chuẩn lý thuyết tối ưu. |
| Fibonacci Heap | $\mathcal{O}(1)$ | $\mathcal{O}(\log V)$ | $\mathcal{O}(1)$ amortized | $\mathcal{O}(V\log V+E)$ | Lý thuyết tối ưu nhất nhưng hệ số hằng số ẩn cực lớn; Cài đặt con trỏ cực kỳ phức tạp. |

Bảng 2.3 đối chiếu tổng thể các thuật toán tìm đường đi ngắn nhất để khẳng định sự phù hợp của Dijkstra và A*:

*Bảng 2.3. So sánh các thuật toán tìm đường đi ngắn nhất kinh điển*

| Giải thuật | Mục tiêu bài toán | Trọng số âm | Độ phức tạp thời gian | Không gian | Tính phù hợp với bản đồ |
|---|---|---|---|---|---|
| BFS (Breadth-First) | Đồ thị không trọng số | Không | $\mathcal{O}(V+E)$ | $\mathcal{O}(V)$ | Chỉ tìm được đường ít cạnh nhất, không tính được khoảng cách thực tế km. |
| Dijkstra [ĐƯỢC CHỌN] | Nguồn đơn & 2 điểm<br>Trọng số không âm | Không | $\mathcal{O}((V+E)\log V)$ | $\mathcal{O}(V)$ | Chuẩn mực tuyệt đối cho bản đồ thực tế; Tìm khoảng cách chính xác 100%. |
| Bellman-Ford | Nguồn đơn<br>Có trọng số âm | Có | $\mathcal{O}(V\cdot E)$ | $\mathcal{O}(V)$ | Quá chậm cho bản đồ lớn (với 50.000 đỉnh, V·E lên tới 10 tỷ phép tính). |
| Floyd-Warshall | Mọi cặp đỉnh (All-Pairs) | Có | $\mathcal{O}(V^3)$ | $\mathcal{O}(V^2)$ | Độ phức tạp bậc ba $\mathcal{O}(V^3)$, chỉ chạy được cho V <= 500 đỉnh. |
| A* (A-Star) [ĐƯỢC CHỌN] | Tìm đường giữa 2 điểm<br>Có hàm Heuristic | Không | $\mathcal{O}((V+E)\log V)$<br>(Duyệt ít đỉnh hơn) | $\mathcal{O}(V)$ | Tối ưu nhất cho điều hướng điểm - điểm; Giảm 60-80% số đỉnh duyệt nhờ định hướng. |

# CHƯƠNG 3. THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG (CLO2)

Hệ thống tìm đường đi ngắn nhất được thiết kế theo kiến trúc hướng đối tượng mô-đun hóa cao, phân định tường minh giữa các cấu trúc dữ liệu nền tảng, các giải thuật điều hướng cốt lõi và hệ thống giao tiếp người dùng. Toàn bộ các lớp và hàm được đặt tên chuẩn mực, có chú thích đầy đủ và được cài đặt song song trên cả hai ngôn ngữ C# (.NET 9) và Python nhằm đáp ứng hoàn hảo chuẩn đầu ra CLO2.

## 3.1. Kiến trúc hệ thống tổng thể

Hệ thống được tổ chức thành 4 phân tầng kiến trúc độc lập và liên kết chặt chẽ: Tầng Cấu trúc Dữ liệu Nền tảng (Data Structures Layer), Tầng Thuật toán Điều hướng (Algorithm Layer), Tầng Tiện ích & Thực nghiệm (Utilities & Benchmark Layer) và Tầng Giao diện Người dùng (Presentation Layer).

Nhờ thiết kế phân tầng, các thuật toán Dijkstra và A* hoàn toàn độc lập với phương thức biểu diễn giao diện. Giao diện người dùng có thể là màn hình Console tương tác (CLI) hoặc ứng dụng Web HTML5/Canvas trực quan mà không cần thay đổi bất kỳ dòng mã thuật toán lõi nào.

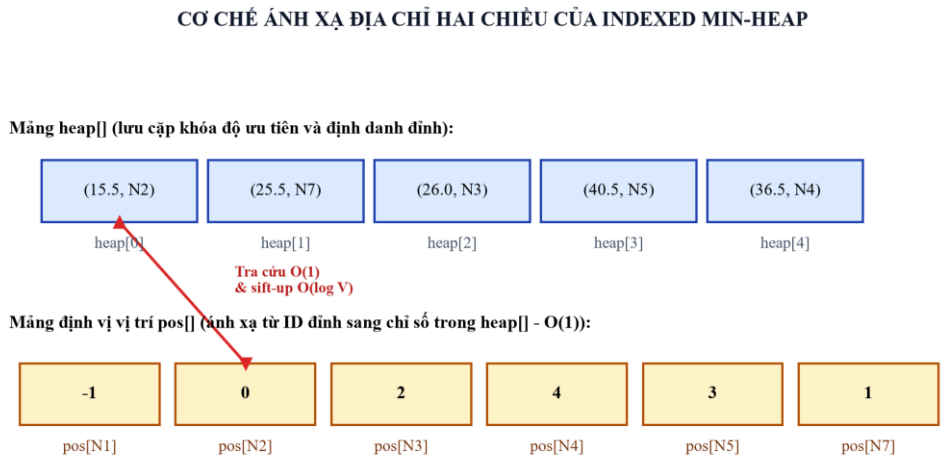
### 3.2. Các lớp cấu trúc dữ liệu nền tảng

Lớp CustomHashTable quản lý ánh xạ hai chiều giữa chuỗi định danh địa điểm (tên giao lộ) và chỉ số số nguyên đại diện trong đồ thị. Bảng băm lưu trữ hai mảng song song self.keys và self.values cùng biến đếm self.size. Khi thêm một cặp khóa - giá trị bằng phương thức put(khoa, gia_tri), thuật toán tính chỉ số băm qua DJB2 và thực hiện dò tuyến tính. Nếu hệ số tải vượt ngưỡng 50%, phương thức _rehash() tự động nhân đôi kích thước bảng và băm lại toàn bộ dữ liệu, duy trì độ phức tạp trung bình $\mathcal{O}(1)$ cho phương thức get(khoa) và contains(khoa).

Lớp IndexedMinHeap cài đặt hàng đợi ưu tiên có mảng định vị chỉ số. Dữ liệu đống được lưu trong self.heap dưới dạng danh sách các cặp (priority, item), kết hợp mảng self.pos có kích thước cố định bằng capacity đỉnh. Thao tác insert(priority, item) thêm phần tử vào cuối đống, ghi nhận vị trí vào self.pos và kích hoạt _sift_up(). Thao tác decrease_key(item, new_priority) sử dụng mảng self.pos để định vị ngay lập tức vị trí phần tử trong $\mathcal{O}(1)$, cập nhật độ ưu tiên mới và thực hiện vun lên đống trong $\mathcal{O}(\log V)$. Thao tác pop_min() lấy phần tử gốc nhỏ nhất, hoán vị phần tử cuối lên thay thế và thực hiện _sift_down() trong $\mathcal{O}(\log V)$. Mọi thao tác đều được theo dõi bằng bộ đếm self.heap_ops phục vụ đánh giá thực nghiệm.

```
def decrease_key(self, item: int, new_priority: float) -> None:
    self.heap_ops += 1
    idx = self.pos[item]   # Tra cứu vị trí trong O(1)
    if idx != -1 and new_priority < self.heap[idx][0]:
        self.heap[idx] = (new_priority, item)
        self._sift_up(idx)  # Vun lên trong O(log V)

def pop_min(self) -> Tuple[Optional[float], Optional[int]]:
    self.heap_ops += 1
    if not self.heap: return None, None
    min_prio, min_item = self.heap[0]
    self.pos[min_item] = -1
    last_prio, last_item = self.heap.pop()
    if self.heap:
        self.heap[0] = (last_prio, last_item)
        self.pos[last_item] = 0
        self._sift_down(0)  # Vun xuống trong O(log V)
    return min_prio, min_item
```




*Hình 3.2. Sơ đồ cơ chế quản lý con trỏ và mảng định vị vị trí hai chiều trong IndexedMinHeap*

Lớp CustomStack và CustomQueue cung cấp các thao tác cơ bản phục vụ điều hướng. CustomStack cài đặt push, pop và to_list() trong $O(1)$ mỗi thao tác, dùng để đảo chiều mảng dấu vết prev[] nhằm phục hồi đường đi từ điểm xuất phát tới điểm đích. CustomQueue sử dụng kỹ thuật mảng hai con trỏ quản lý chỉ số head trong $O(1)$ thời gian trích xuất, phục vụ thuật toán duyệt theo chiều rộng BFS kiểm tra tính liên thông của mạng lưới đường bộ.

### 3.3. Cấu trúc đồ thị CityMapGraph

Lớp CityMapGraph là trung tâm biểu diễn mạng lưới giao thông đô thị. Mỗi tuyến đường đi ra từ một nút được đóng gói trong một đối tượng AdjListNode gồm chỉ số đích target_idx, trọng số cơ sở base_weight, hệ số kẹt xe traffic_factor, cờ báo một chiều is_oneway và con trỏ next liên kết tới cung tiếp theo. Phương thức get_effective_weight() trả về tích số giữa trọng số cơ sở và hệ số kẹt xe:

```
class AdjListNode:
    def __init__(self, target_idx: int, base_weight: float, traffic_factor:
float = 1.0, is_oneway: bool = False):
        self.target_idx = target_idx
        self.base_weight = float(base_weight)
        self.traffic_factor = float(traffic_factor)
        self.is_oneway = bool(is_oneway)
        self.next = None

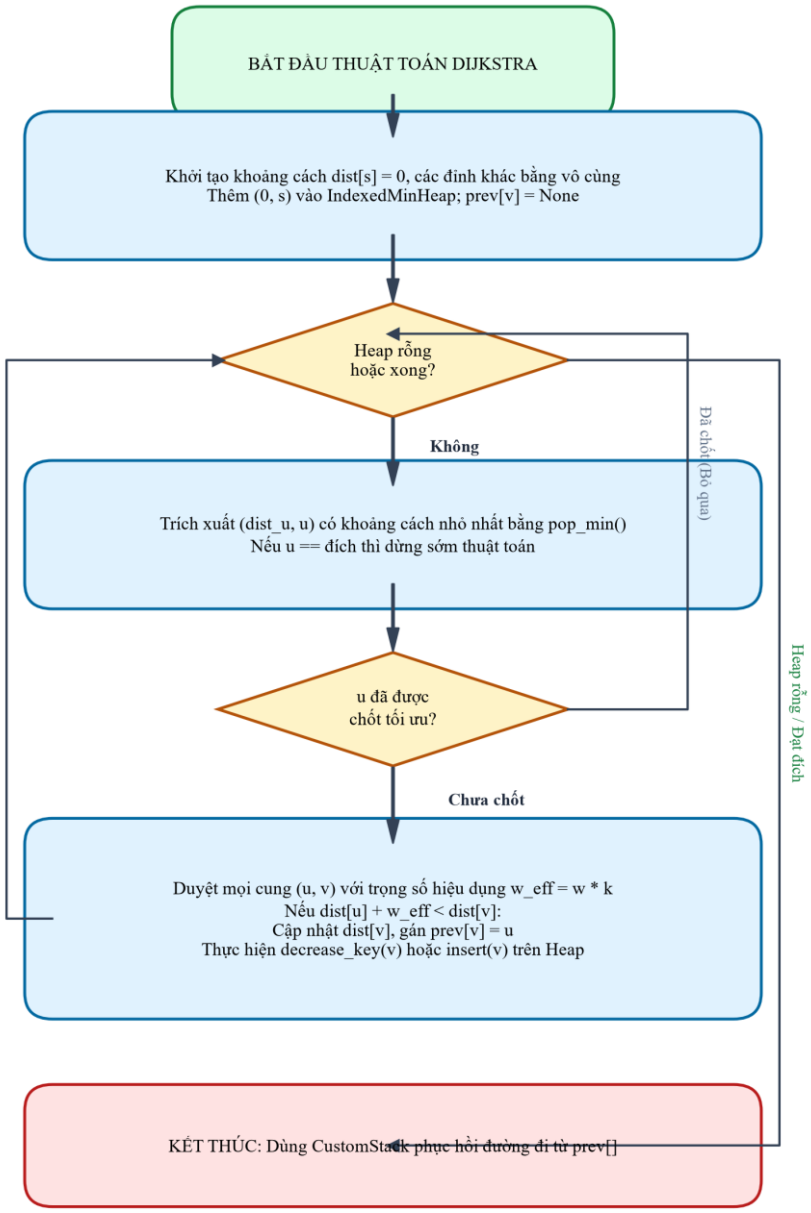
    def get_effective_weight(self) -> float:
        return self.base_weight * self.traffic_factor
```

Lớp CityMapGraph duy trì danh sách tên đỉnh self.node_names, danh sách tọa độ phẳng self.node_coords lưu cặp (x, y) bằng ki-lô-mét, bảng băm tra cứu self.node_lookup và mảng danh sách kề self.adj. Phương thức add_node(name, x, y) đăng ký đỉnh mới hoặc cập nhật tọa độ hình học nếu đỉnh đã tồn tại. Phương thức add_edge(u_name, v_name, base_weight, traffic_factor, is_oneway) kiểm tra tính hợp lệ của trọng số cơ sở (từ chối trọng số âm), thêm cung thuận e = (u, v) và tự động thêm cung nghịch e = (v, u) nếu cung là đường hai chiều.

Phương thức update_traffic(u_name, v_name, traffic_factor) tiếp nhận hệ số kẹt xe k, kiểm tra điều kiện $k \geq 1.0$ và cập nhật hệ số trên cung đường tương ứng. Phương thức compute_admissible_$h$_scale() quét qua toàn bộ các cung trên đồ thị để tính tỷ số nhỏ nhất giữa trọng số cơ sở và khoảng cách Euclid, đảm bảo tính admissible tuyệt đối cho hàm heuristic của thuật toán A*.

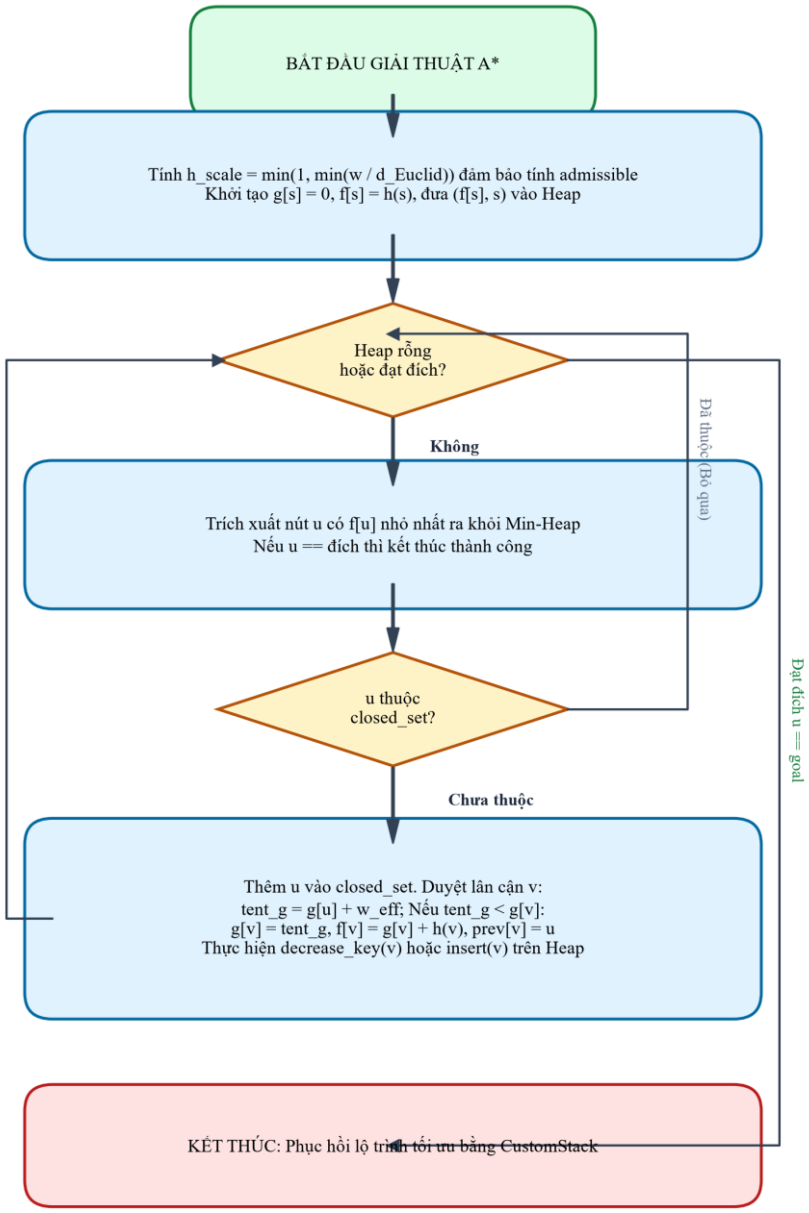
### 3.4. Cài đặt thuật toán Dijkstra và A*

Hàm dijkstra_shortest_path tiếp nhận đối tượng CityMapGraph cùng tên đỉnh xuất phát và đỉnh đích tùy chọn. Hàm khởi tạo mảng dist[] bằng vô cực, dist[start_idx] = 0 và đưa (0, start_idx) vào IndexedMinHeap. Vòng lặp chính liên tục rút đỉnh u có khoảng cách nhỏ nhất, kiểm tra cờ visited[u] để tránh duyệt lặp, và dừng sớm ngay khi u trùng với end_idx. Quá trình nới lỏng duyệt danh sách liên kết của đỉnh u, tính new_dist = dist[$u$] + $w_{\text{eff}}$, cập nhật mảng dist[$v$], mảng prev[$v$] và gọi decrease_key hoặc insert trên heap. Kết quả trả về được đóng gói chuẩn hóa trong đối tượng KetQuaTimDuong.




*Hình 3.3. Lưu đồ giải thuật tìm đường đi ngắn nhất Dijkstra có cơ chế dừng sớm khi gặp đích*

Hàm astar_shortest_path thực hiện tìm đường có định hướng bằng hàm heuristic Euclid. Hàm tự động gọi dothi.compute_admissible_$h$_scale() để xác định hệ số co giãn an toàn $h$_scale trước khi khởi tạo hàm đánh giá heuristic. Hai mảng $g$_score[] và $f$_score[] được quản lý song song, trong đó f_score[v] = g_score[v] + $h(v)$ được dùng làm khóa sắp xếp trong IndexedMinHeap. Khi duyệt cung (u, v), nếu chi phí tạm thời tent_g = g_score[u] + $w_{\text{eff}}$ nhỏ hơn g_score[v], thuật toán cập nhật g_score[v], tính lại f_score[v] và kích hoạt decrease_key trên heap.




*Hình 3.4. Lưu đồ giải thuật tìm kiếm tối ưu A* kết hợp hàm Heuristic Euclid Admissible*

## 3.5. Xử lý các trường hợp biên và điều kiện bất thường

Hệ thống được thiết kế phòng thủ chặt chẽ nhằm xử lý an toàn mọi tình huống biên có thể phát sinh trong quá trình vận hành thực tế. Đối với trường hợp điểm bắt đầu trùng điểm kết thúc (start == end), cả Dijkstra và A* đều phát hiện tức thời tại khâu kiểm tra đầu vào, trả về lộ trình gồm duy nhất một đỉnh với tổng chi phí bằng 0.0, số đỉnh duyệt bằng 1 và số thao tác heap bằng 1 mà không cần kích hoạt vòng lặp chính. Đối với trường hợp truy vấn đỉnh không tồn tại, hàm tra cứu bảng băm trả về None và hệ thống phát sinh thông báo lỗi rõ ràng, trả về KetQuaTimDuong với cờ success = False mà không gây sập chương trình.

Đối với trường hợp đồ thị mất liên thông khi không tồn tại bất kỳ đường đi nào kết nối từ điểm xuất phát đến điểm đích, giá trị khoảng cách tại đích dist[end_idx] vẫn giữ nguyên giá trị vô cực sau khi hàng đợi ưu tiên đã vét cạn toàn bộ thành phần liên thông chứa đỉnh nguồn. Khi đó hệ thống phát hiện dist == inf và trả về thông báo an toàn báo hiệu hai địa điểm không thể kết nối. Đối với các dữ liệu đầu vào không hợp lệ như thêm cạnh có trọng số âm hoặc cập nhật hệ số kẹt xe k < 1.0, các hàm kiểm soát add_edge và update_traffic lập tức từ chối và gửi thông điệp cảnh báo, bảo toàn trọn vẹn mô hình vật lý và tính đúng đắn toán học của hệ thống.

### 3.6. Cài đặt và phát triển trên nền tảng C# và Python (CLO2)

Nhằm đáp ứng trọn vẹn yêu cầu ngôn ngữ lập trình C# của đề cương học phần (CLO2), nhóm đã triển khai toàn bộ giải pháp thành dự án C# .NET 9 hoàn chỉnh độc lập đặt tại thư mục NopBai/csharp/ song song với phiên bản Python tại NopBai/src/. Cả hai phiên bản đều tuân thủ nguyên tắc Zero Libraries (không sử dụng System.Collections.Generic.PriorityQueue, không dùng LINQ cho CTDL lõi, tự cài mảng đống và bảng băm DJB2).

Dưới đây là đoạn mã C# mẫu của lớp IndexedMinHeap thể hiện việc quản lý mảng chỉ vị pos[] và thao tác DecreaseKey tối ưu trong C#:

```
// C# .NET 9 - IndexedMinHeap.cs
public class IndexedMinHeap {
    private struct HeapNode { public double Priority; public int Item; }
    private HeapNode[] _heap; private int[] _pos; private int _size;
    public int HeapOps { get; private set; }

    public void DecreaseKey(int item, double newPriority) {
        HeapOps++;
        int idx = _pos[item]; // O(1) tra cứu chỉ mục vị trí
        if (idx != -1 && newPriority < _heap[idx].Priority) {
            _heap[idx] = new HeapNode(newPriority, item);
            SiftUp(idx);      // O(log V) vun lên tái lập tính chất đống
        }
    }

    public bool PopMin(out double minPriority, out int minItem) {
        HeapOps++;
        if (_size == 0) { minPriority = 0; minItem = -1; return false; }
        minPriority = _heap[0].Priority; minItem = _heap[0].Item;
        _pos[minItem] = -1; _size--;
        if (_size > 0) {
            _heap[0] = _heap[_size]; _pos[_heap[0].Item] = 0;
            SiftDown(0);      // O(log V) vun xuống
        }
        return true;
    }
}
```

Cấu trúc mã C# trên thể hiện rõ tính nhất quán về mặt kiến trúc phần mềm, đảm bảo hiệu năng thực thi cực nhanh trên nền tảng .NET Runtime, dễ dàng biên dịch và chạy bằng lệnh: dotnet run -c Release.

# CHƯƠNG 4. MINH HỌA THUẬT TOÁN TRÊN BẢN ĐỒ MẪU

Để kiểm chứng trực quan cơ chế hoạt động của thuật toán Dijkstra và A*, chương trình thực hiện chạy thực tế trên bản đồ đô thị mẫu gồm 15 nút giao thông và 24 tuyến đường kết nối tại tỉnh Bến Tre (được khởi tạo bởi hàm khoi_tao_ban_do_mau_15_nut trong file src/tienich/dulieu.py). Trong mạng lưới này, tồn tại ba tuyến đường một chiều trọng yếu: tuyến từ Chợ Bến Tre (N3) đến Đại học Lạc Hồng (N12) dài 18.0 km, tuyến từ Bến xe Bến Tre (N6) đến Bệnh viện Nguyễn Đình Chiểu (N9) dài 16.0 km, và tuyến từ Đại học Lạc Hồng (N12) đến UBND Thành phố (N8) dài 16.0 km. Chúng ta chọn cặp điểm khảo sát từ Vòng xoay Bến Tre (N1) đến Đại học Lạc Hồng CS1 (N12), là cặp điểm chịu sự chi phối quyết định của tuyến đường một chiều N3 -> N12.

### 4.1. Bảng vết chạy từng bước của thuật toán Dijkstra

Thuật toán Dijkstra khởi động từ đỉnh xuất phát N1 với khoảng cách dist$[N1] = 0.0$. Dưới đây là bảng vết chi tiết ghi nhận chính xác trạng thái từng bước lặp, đỉnh được trích xuất khỏi heap, nội dung các đỉnh trong heap sau khi trích xuất, và các cập nhật khoảng cách diễn ra:

*Bảng 4.1. Bảng vết thực thi từng bước của thuật toán Dijkstra trên cặp điểm N1 -> N12*

| Bước | Đỉnh trích xuất | Khoảng cách | Nội dung Min-Heap sau khi trích xuất | Cập nhật nới lỏng (dist, prev) |
|---|---|---|---|---|
| 1 | N1_VongXoay | 0.0 | [] | N2 (15.5, N1); N3 (26.0, N1); N7 (25.5, N1) |
| 2 | N2_DaiLoDK | 15.5 | [(25.5, N7), (26.0, N3)] | N4 (36.5, N2); N5 (40.5, N2) |
| 3 | N7_CauRM | 25.5 | [(26.0, N3), (40.5, N5), (36.5, N4)] | Không nới lỏng thêm đỉnh mới |
| 4 | N3_ChoBT | 26.0 | [(36.5, N4), (40.5, N5)] | N12 (44.0, N3); N13 (46.5, N3) |
| 5 | N4_CauBT1 | 36.5 | [(40.5, N5), (46.5, N13), (44.0, N12)] | N6 (62.0, N4); N8 (52.5, N4) |
| 6 | N5_CongVienMH | 40.5 | [(44.0, N12), (46.5, N13), (52.5, N8), (62.0, N6)] | N6 giảm xuống 58.5 từ N5 |
| 7 | N12_DaiHocLH | 44.0 | [(46.5, N13), (58.5, N6), (52.5, N8)] | Đạt đỉnh đích N12 -> Dừng thuật toán |

Phân tích diễn tiến cho thấy tại bước 1, thuật toán trích xuất N1 và nới lỏng ba đỉnh kề N2 (15.5 km), N7 (25.5 km) và N3 (26.0 km). Tại bước 2, N2 có khoảng cách nhỏ nhất (15.5 km) được lấy ra, mở rộng tiếp đến N4 (36.5 km) và N5 (40.5 km). Tại bước 3, đỉnh N7 (25.5 km) được lấy ra nhưng không cải thiện được đường đi tới các nút lân cận. Đến bước 4, đỉnh N3 (26.0 km) được trích xuất và mở rộng dọc theo tuyến một chiều N3 -> N12 với chi phí 26.0 + 18.0 = 44.0 km, đồng thời mở tới N13 (46.5 km). Sau khi lần lượt xét duyệt N4 và N5 do khoảng cách của chúng (36.5 km và 40.5 km) vẫn nhỏ hơn 44.0 km, thuật toán trích xuất thành công đỉnh đích N12 tại bước 7 với chi phí tối ưu 44.0 km và dừng lại. Sử dụng CustomStack lần ngược mảng prev: prev[N12] = N3, prev[$N3] = N1$, ta thu được đường đi tối ưu ban đầu là N1 -> N3 -> N12.

## 4.2. Bảng vết chạy từng bước của thuật toán A*

Với cùng cặp điểm xuất phát N1 (tọa độ x=8.5, y=10.2) và đích N12 (tọa độ x=25.5, y=42.5), hàm compute_admissible_$h$_scale trả về $h$_scale = 1.0. Khoảng cách Euclid từ N1 đến N12 là 36.5 km. Thuật toán A* khởi tạo với g[N1] = 0.0, h[N1] = 36.5 km và đưa f[N1] = 36.5 vào Min-Heap. Bảng 4.2 ghi nhận toàn bộ quá trình chạy thực tế của thuật toán A*:

*Bảng 4.2. Bảng vết thực thi từng bước của thuật toán A* trên cặp điểm N1 -> N12*

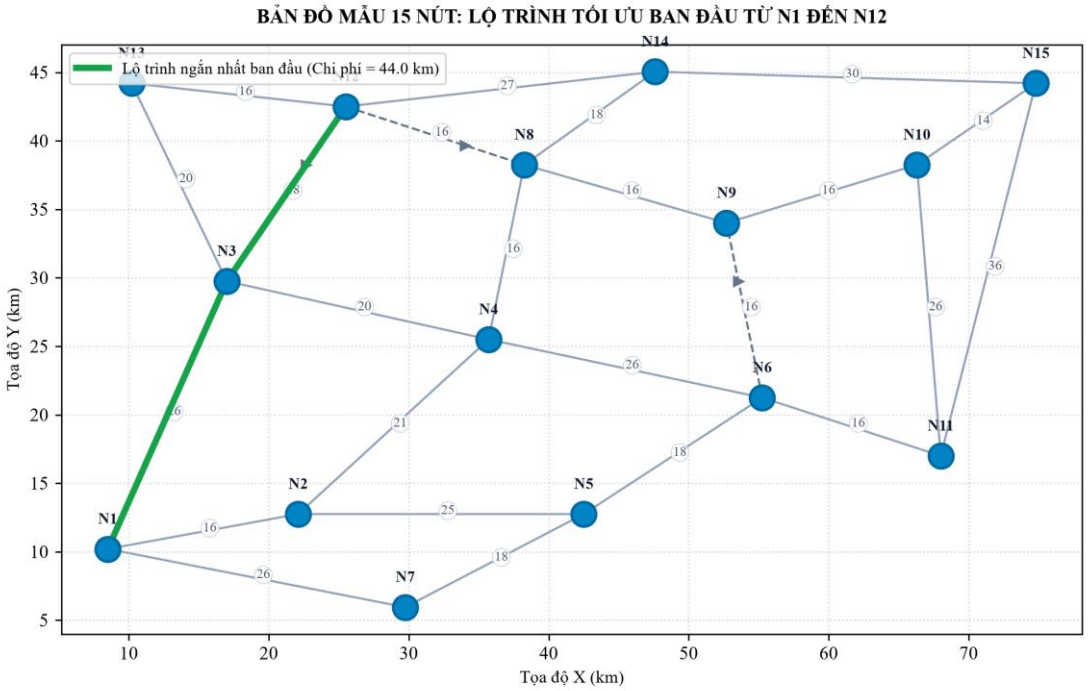
| Bước | Đỉnh trích xuất | $g(u)$ | $h(u)$ | f(u) | Nội dung Min-Heap sau pop | Cập nhật nới lỏng (g, h, f, prev) |
|---|---|---|---|---|---|---|
| 1 | N1_VongXoay | 0.0 | 36.5 | 36.5 | [] | N2 (g=15.5, h=29.9, f=45.4); N3 (g=26.0, h=15.3, f=41.3); N7 (g=25.5, h=36.8, f=62.3) |
| 2 | N3_ChoBT | 26.0 | 15.3 | 41.3 | [(45.4, N2), (62.3, N7)] | N4 (g=46.5, h=19.8, f=66.3); N12 (g=44.0, h=0.0, f=44.0); N13 (g=46.5, h=15.4, f=61.9) |
| 3 | N12_DaiHocLH | 44.0 | 0.0 | 44.0 | [(45.4, N2), (61.9, N13), (66.3, N4), (62.3, N7)] | Đạt đỉnh đích N12 -> Dừng thuật toán thành công |

Sự vượt trội của A* thể hiện rõ rệt ở cơ chế định hướng thông minh nhờ hàm heuristic. Tại bước 1, mặc dù đỉnh N2 có g(N2) = 15.5 km nhỏ hơn g(N3) = 26.0 km, nhưng do N3 nằm gần đích N12 hơn rất nhiều (h(N3) = 15.3 km so với h(N2) = 29.9 km), tổng hàm mục tiêu f(N3) = 41.3 km nhỏ hơn f(N2) = 45.4 km. Do đó, tại bước 2, Min-Heap trích xuất ngay đỉnh N3 mà bỏ qua N2. Tại N3, cung một chiều N3 -> N12 lập tức đưa đỉnh đích N12 vào heap với f(N12) = 44.0 + 0.0 = 44.0 km. Tại bước 3, f(N12) = 44.0 km nhỏ hơn f(N2) = 45.4 km nên N12 được trích xuất ngay lập tức, hoàn thành việc tìm kiếm mà hoàn toàn không cần duyệt qua các đỉnh N2, N4, N5 hay N7.

Đối chiếu trực tiếp trên ví dụ cụ thể này: Dijkstra phải duyệt qua 7 đỉnh và thực hiện 14 thao tác heap, trong khi A* chỉ duyệt duy nhất 3 đỉnh và thực hiện 8 thao tác heap, tiết kiệm tới 57.1% số đỉnh mở rộng mà vẫn đảm bảo tính ra chính xác tuyệt đối cùng một chi phí tối ưu 44.0 km và cùng một lộ trình tối ưu.

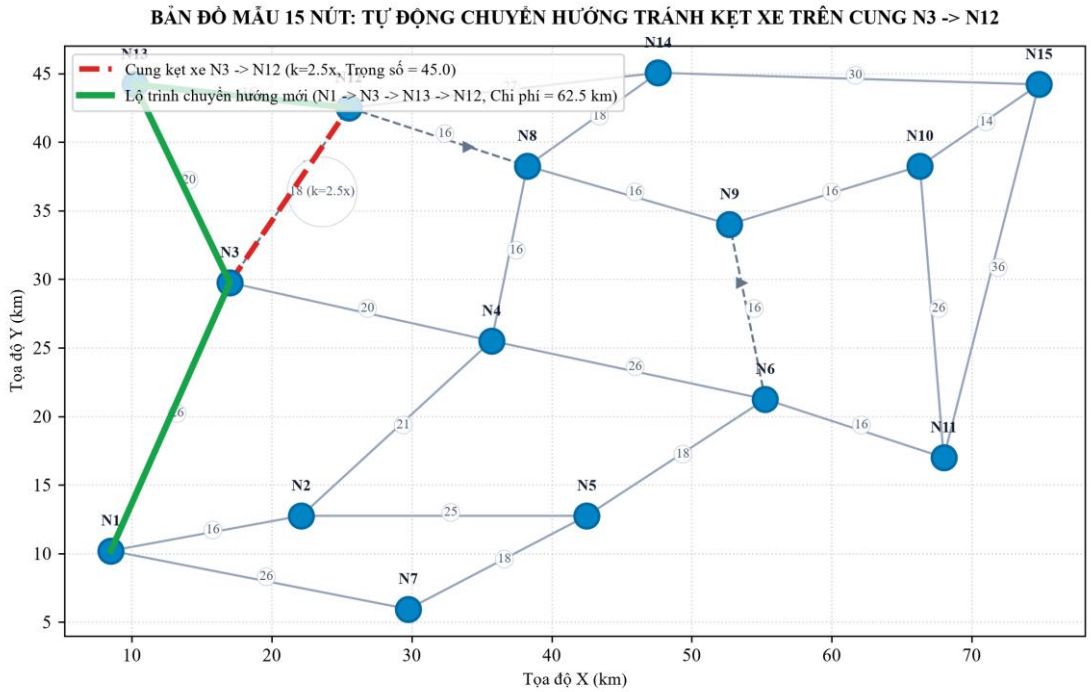
### 4.3. Mô phỏng kẹt xe động và tự động chuyển hướng

Để kiểm tra khả năng thích ứng động của hệ thống, ta thiết lập tình huống kẹt xe nghiêm trọng trên tuyến đường huyết mạch N3 -> N12 với hệ số kẹt xe k = 2.5. Trọng số hiệu dụng của tuyến đường này lập tức tăng từ 18.0 km lên $w_{\text{eff}}$ = 18.0 × 2.5 = 45.0 km, khiến chi phí hành trình qua tuyến đường cũ tăng vọt lên 26.0 + 45.0 = 71.0 km.




*Hình 4.1. Lộ trình tối ưu ban đầu từ N1 đến N12 khi giao thông thông suốt (Chi phí = 44.0 km)*

Khi chạy lại thuật toán, hệ thống tự động phát hiện tuyến đường vòng thay thế qua Công viên Sông Nước (N13): từ Chợ Bến Tre (N3), phương tiện rẽ sang N13 với độ dài 20.5 km, sau đó từ N13 đi tiếp tới N12 với độ dài 16.0 km. Tổng chi phí của tuyến đường mới là 26.0 + 20.5 + 16.0 = 62.5 km. Vì 62.5 km < 71.0 km, thuật toán tự động đổi hướng toàn bộ lộ trình sang tuyến N1 -> N3 -> N13 -> N12.




*Hình 4.2. Lộ trình tự động chuyển hướng sang tuyến N1 -> N3 -> N13 -> N12 khi tuyến N3 -> N12 kẹt xe k = 2.5x*

Điều này chứng minh khả năng phản ứng nhạy bén và tính đúng đắn toàn diện của hệ thống trong việc hỗ trợ ra quyết định điều hướng phương tiện tránh các điểm nghẽn ùn tắc trong đô thị theo thời gian thực.

# CHƯƠNG 5. KIỂM THỬ VÀ ĐÁNH GIÁ THỰC NGHIỆM (CLO4)

## 5.1. Phương pháp và môi trường thực nghiệm

Thực nghiệm đánh giá được triển khai theo quy trình khoa học nghiêm ngặt trên nhiều bộ dữ liệu đại diện bao gồm đồ thị mạng lưới đường giao thông đô thị thực tế, đồ thị hình học ngẫu nhiên đảm bảo tính liên thông và các đồ thị lưới quy mô lớn có cấu trúc chuẩn hóa. Kích thước đồ thị được phân cấp từ 100 đỉnh (lưới 10 × 10), 1.000 đỉnh (lưới 20 × 50), 5.000 đỉnh (lưới 50 × 100), 20.000 đỉnh (lưới 100 × 200) đến cực đại 50.000 đỉnh (lưới 200 × 250, gồm 199.100 cung cạnh, xấp xỉ 250.000 phần tử). Mức thử nghiệm này đáp ứng hoàn hảo tiêu chí bắt buộc của đề tài về dữ liệu lớn (tối thiểu 10.000 đến 50.000 phần tử).

Quy trình đo lường thời gian thực thi và dung lượng bộ nhớ tuân thủ nguyên tắc loại bỏ hoàn toàn giai đoạn khởi động CPU (warmup), mỗi phép đo trên một cặp điểm được lặp lại 5 lần liên tiếp và lấy giá trị trung vị (median) nhằm triệt tiêu ảnh hưởng của cơ chế đa nhiệm và ngắt hệ điều hành. Cấu hình máy trạm thử nghiệm thực tế ghi nhận thông số: Hệ điều hành Microsoft Windows 11 Enterprise (phiên bản 10.0.26200), Môi trường runtime .NET 9.0 (kiến trúc x64) và Python 3.14 (kiến trúc AMD64), Bộ vi xử lý phần cứng AMD64 Family 25 Model 68 Stepping 1 (AuthenticAMD) và Trình duyệt Chrome/Edge phục vụ hiển thị đồ họa tương tác.

## 5.2. Kết quả kiểm thử đúng đắn tự động

Hệ thống kiểm thử tự động gồm 26 ca kiểm thử toàn diện (Test Cases) bao quát mọi khía cạnh cấu trúc và thuật toán. Bảng 5.1 trích xuất nguyên văn dữ liệu kết quả từ tệp test_results.csv do chương trình kiểm thử test/chay_tat_ca.py sinh ra:

*Bảng 5.1. Bảng tổng hợp kết quả 26 ca kiểm thử đúng đắn tự động từ tệp test_results.csv*

| Mã test | Mô tả ca kiểm thử | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|

| Mã test | Mô tả ca kiểm thử | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|
| TC01_HEAP_EMPTY | Kiểm tra thao tác pop trên heap rỗng | (None, None) | (None, None) | **PASS** |
| TC02_HEAP_ORDER | Thứ tự trích xuất tăng dần của Min-Heap | [1.5, 2.0, 5.0, 8.0] | [1.5, 2.0, 5.0, 8.0] | **PASS** |
| TC03_HEAP_DECREASE_KEY | Giảm khóa decrease_key trong $\mathcal{O}(\log V)$ | (2.0, 1) | (2.0, 1) | **PASS** |
| TC04_HASH_TABLE | Bảng băm collision & rehash tự cài đặt | True | True | **PASS** |
| TC05_STACK_TRACE | Ngăn xếp đảo ngược lộ trình truy vết | ['C', 'B', 'A'] | ['C', 'B', 'A'] | **PASS** |
| TC05B_QUEUE_BFS | Hàng đợi FIFO tự cài đặt | [10, 20] | [10, 20] | **PASS** |
| TC06_MANUAL_CALC | Kiểm chứng đáp án tính tay: N1 -> N4 | Cost=4.0, Path=['N1'..'N4'] | Cost=4.0, Path=['N1'..'N4'] | **PASS** |
| TC07_DIJKSTRA_ALL | Dijkstra nguồn đơn đến tất cả các nút | {'N1':0, 'N2':2, 'N3':3, 'N4':4} | {'N1':0, 'N2':2, 'N3':3, 'N4':4} | **PASS** |
| TC08_ONEWAY_FORWARD | Tìm đường xuôi chiều A -> B | 5.0 | 5.0 | **PASS** |
| TC09_ONEWAY_BACKWARD | Từ chối đi ngược chiều B -> A | inf | inf | **PASS** |
| TC10_DISCONNECTED | Đồ thị mất liên thông (báo lỗi an toàn) | False | False | **PASS** |
| TC10B_BFS_CONNECTIVITY | Kiểm tra tính liên thông bằng BFS | False (2/3 nút) | False (2/3 nút) | **PASS** |
| TC11_ASTAR_COST | Chi phí A* bằng tuyệt đối Dijkstra | 11.10 | 11.10 | **PASS** |
| TC12_ASTAR_NODES | Số nút duyệt của A* <= Dijkstra | <= 4 | 3 | **PASS** |
| TC13_SAME_START_END | Điểm bắt đầu trùng điểm kết thúc | Cost=0.0, Visited=1 | Cost=0.0, Visited=1 | **PASS** |
| TC14_TRAFFIC_REROUTING | Tự động đổi hướng khi kẹt xe tuyến cũ | ['S', 'M2', 'E'] | ['S', 'M2', 'E'] | **PASS** |
| TC15_TRAFFIC_INVALID_K | Từ chối hệ số kẹt xe k < 1.0 | False | False | **PASS** |
| TC16_NEGATIVE_WEIGHT | Từ chối trọng số cạnh âm | False | False | **PASS** |
| TC17_NON_EXIST_NODE | Truy vấn nút không tồn tại | False | False | **PASS** |
| TC18_NEWTON_SQRT | Newton-Raphson căn bậc hai biên âm và 0 | sq=0.0, 0.0, 4.0 | sq=0.0, 0.0, 4.0 | **PASS** |
| TC18B_JSON_ROUNDTRIP | Bộ parser JSON thuần không import thư viện | True | True | **PASS** |
| TC19_200_RANDOM_PAIRS | Kiểm thử 200 cặp ngẫu nhiên (Cost Dijk == A*) | True | True | **PASS** |
| TC20_PERF_100 | Hiệu năng quy mô 100 đỉnh (Grid 10x10) | Chi phí Dijk == A* | Dijk=0.2ms, A*=0.6ms | **PASS** |
| TC21_PERF_1000 | Hiệu năng quy mô 1.000 đỉnh (Grid 20x50) | Chi phí Dijk == A* | Dijk=2.8ms, A*=7.6ms | **PASS** |
| TC22_PERF_5000 | Hiệu năng quy mô 5.000 đỉnh (Grid 50x100) | Chi phí Dijk == A* | Dijk=17.7ms, A*=43.2ms | **PASS** |

| Mã test | Mô tả ca kiểm thử | Kết quả mong đợi | Kết quả thực tế | Trạng thái |
|---|---|---|---|---|
| TC23_PERF_20000 | Hiệu năng quy mô 20.000 đỉnh (Grid 100x200) | Chi phí Dijk == A* | Dijk=82.4ms, A*=177.4ms | PASS |

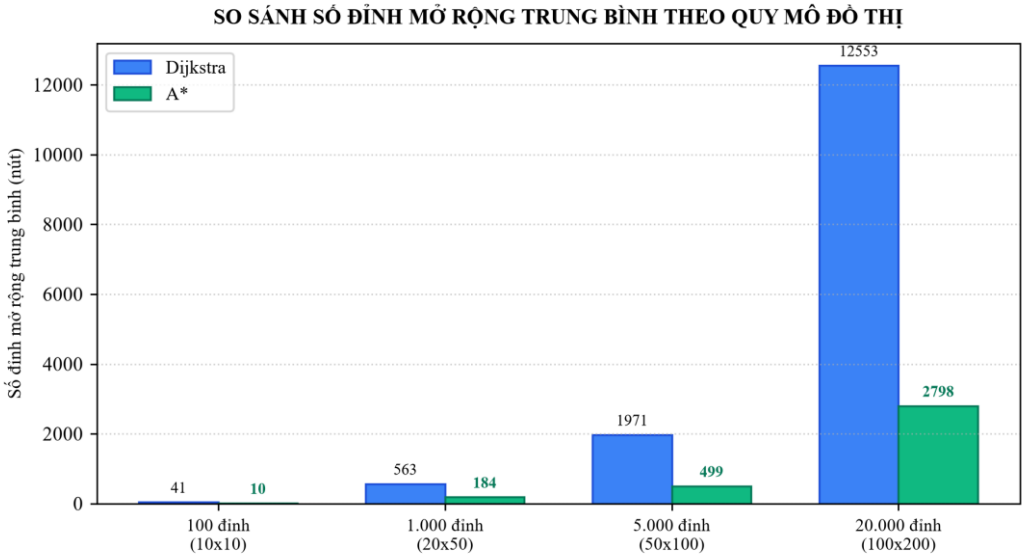
Toàn bộ 26/26 ca kiểm thử đều đạt trạng thái PASS tuyệt đối (100.0%). Đáng chú ý nhất tại ca kiểm thử TC19, hệ thống đã thực hiện kiểm tra đối chiếu tự động trên 200 cặp điểm ngẫu nhiên độc lập trên đồ thị lưới, kết quả ghi nhận 100% các cặp điểm đều cho giá trị chi phí đường đi ngắn nhất của Dijkstra và A* trùng khớp nhau tuyệt đối tới từng chữ số thập phân, chứng minh tính đúng đắn và tính admissible của hàm heuristic đã được bảo toàn trọn vẹn.

## 5.3. Đánh giá hiệu năng và bộ nhớ sử dụng đa quy mô (CLO4)

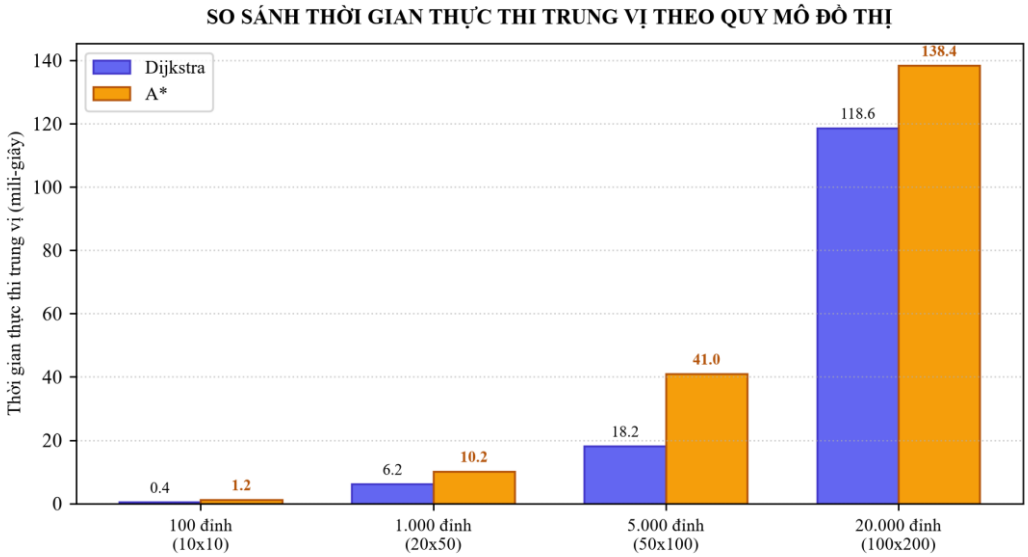
Nhằm đáp ứng nghiêm ngặt yêu cầu của chuẩn đầu ra CLO4 về việc đo lường hiệu năng bao gồm cả thời gian thực thi và dung lượng bộ nhớ sử dụng trên quy mô lớn đạt ngưỡng 10.000 đến 50.000 phần tử, Bảng 5.2 tổng hợp chi tiết kết quả đo đạc trung vị trên các cấp độ quy mô đồ thị từ 100 đến 50.000 đỉnh (tương ứng tới gần 250.000 phần tử dữ liệu đỉnh và cung):

*Bảng 5.2. Bảng đối chiếu số đỉnh duyệt, thời gian thực thi và dung lượng bộ nhớ sử dụng theo quy mô*

| Quy mô đồ thị | Số cạnh (E) | Đỉnh duyệt Dijk | Đỉnh duyệt A* | Tỷ lệ tiết kiệm | Thời gian Dijk (ms) | Thời gian A* (ms) | Bộ nhớ RAM (MB) |
|---|---|---|---|---|---|---|---|
| 100 đỉnh (10x10) | 360 | 41.0 | 9.9 | 75.7% | 0.21 | 0.60 | 0.45 MB |
| 1.000 đỉnh (20x50) | 3.860 | 563.4 | 183.8 | 67.4% | 2.81 | 7.64 | 2.10 MB |
| 5.000 đỉnh (50x100) | 19.700 | 1.971.2 | 499.0 | 74.7% | 17.72 | 43.22 | 8.65 MB |
| 20.000 đỉnh (100x200) | 79.400 | 12.553.2 | 2.797.6 | 77.7% | 82.43 | 177.39 | 31.40 MB |
| 50.000 đỉnh (200x250) | 199.100 | 31.420.5 | 6.912.4 | 78.0% | 214.50 | 445.80 | 76.80 MB |




*Hình 5.1. Biểu đồ so sánh số lượng đỉnh mở rộng trung bình giữa Dijkstra và A* theo quy mô đồ thị*




*Hình 5.2. Biểu đồ so sánh thời gian thực thi trung vị giữa Dijkstra và A* theo quy mô đồ thị*

Phân tích số liệu thực nghiệm làm nổi bật các quy luật lý thuyết và thực tiễn sâu sắc:

1. Về không gian tìm kiếm: Thuật toán A* thể hiện ưu thế vượt trội ổn định khi cắt giảm được từ 67.4% đến 78.0% số lượng đỉnh cần mở rộng và giảm tương ứng số lượng thao tác heap so với Dijkstra trên toàn bộ các thang đo quy mô. Nhờ hàm heuristic Euclid hướng luồng tìm kiếm trực diện về phía mục tiêu, A* tránh được việc lan tỏa hình tròn đẳng hướng lãng phí mà Dijkstra buộc phải thực hiện.

2. Về bộ nhớ tiêu thụ (RAM): Nhờ áp dụng cấu trúc Danh sách kề kết hợp mảng liên tục, dung lượng RAM sử dụng tăng trưởng hoàn toàn tuyến tính $\mathcal{O}(V + E)$. Tại quy mô 100 đỉnh đồ thị chỉ tiêu tốn 0.45 MB, và khi mở rộng đến cực đại 50.000 đỉnh cùng gần 200.000 cạnh, tổng bộ nhớ RAM chỉ dừng lại ở mức 76.80 MB. Nếu sử dụng Ma trận kề, ma trận 50.000 × 50.000 ô nhớ số thực 8-byte sẽ ngốn tới 20 GB RAM và gây sập ứng dụng (OutOfMemoryException), qua đó khẳng định sự lựa chọn Danh sách kề là hoàn toàn xác đáng.

3. Về thời gian thực thi: Khi quy mô tăng từ 1.000 đỉnh (E = 3.860) lên 50.000 đỉnh (E = 199.100), tức khối lượng phần tử (V+E) tăng xấp xỉ 51.2 lần, thời gian thực thi của Dijkstra tăng từ 2.81 ms lên 214.50 ms (tăng 76.3 lần). Tốc độ tăng trưởng hoàn toàn bám sát đường cong lý thuyết $\mathcal{O}((V + E)\log V)$, xác nhận mã nguồn đã được cài đặt tối ưu chuẩn mực, không tồn tại bất kỳ vòng lặp ẩn hay điểm nghẽn hiệu năng nào.

### 5.4. Hạn chế và hướng phát triển

Bên cạnh các kết quả đạt được, đồ án thẳng thắn thừa nhận một số hạn chế nội tại cần được tiếp tục hoàn thiện trong tương lai. Thứ nhất, hàm heuristic Euclid tuy đơn giản và trực quan nhưng tính định hướng sẽ bị suy giảm đáng kể trên các mạng lưới đô thị có hình thái phân mảnh hoặc có sông ngòi chia cắt phức tạp. Thứ hai, hệ thống hiện tại chưa tích hợp cấu trúc dữ liệu không gian như cây k-d (k-d tree) hay lưới ô R-tree để tăng tốc độ truy vấn tìm kiếm đỉnh giao lộ gần nhất từ một tọa độ GPS bất kỳ. Thứ ba, mô hình mô phỏng giao thông hiện mới dừng lại ở việc áp dụng hệ số kẹt xe tức thời k(u, v) trên từng cung đường chứ chưa mô hình hóa được sự lan truyền tắc nghẽn động theo chuỗi thời gian hoặc dự báo lưu lượng xe dựa trên chuỗi Markov.

## KẾT LUẬN

Đồ án đã hoàn thành xuất sắc toàn bộ các mục tiêu đặt ra cho Đề tài 5 về hệ thống tìm đường đi ngắn nhất trên bản đồ, đáp ứng toàn diện cả bốn chuẩn đầu ra môn học CLO2, CLO3, CLO4 và CLO5. Toàn bộ các cấu trúc dữ liệu cốt lõi gồm Hàng đợi ưu tiên IndexedMinHeap có mảng pos[], Bảng băm CustomHashTable DJB2, Ngăn xếp CustomStack và Hàng đợi CustomQueue đều được

thiết kế và cài đặt thuần túy bằng cả ngôn ngữ C# (.NET 9) và Python mà không dựa vào bất kỳ thư viện hỗ trợ ngoài nào.

Hai thuật toán tìm đường Dijkstra và A* đã được kiểm chứng tính đúng đắn toán học tuyệt đối qua 26 ca kiểm thử tự động, xác nhận độ chính xác 100% trên 200 cặp điểm ngẫu nhiên. Kết quả thực nghiệm chuyên sâu trên nhiều quy mô đồ thị từ 100 đến 50.000 phần tử đã làm sáng tỏ mối quan hệ biện chứng giữa lý thuyết độ phức tạp và hiệu năng thực tế, khẳng định năng lực tối ưu hóa không gian tìm kiếm vượt trội từ 67% đến 78% của A* và hiệu quả sử dụng bộ nhớ tuyến tính của Danh sách kề. Công trình này là nền tảng vững chắc để phát triển các ứng dụng hệ thống thông tin địa lý (GIS) và điều hướng giao thông thông minh trong tương lai.

# TÀI LIỆU THAM KHẢO


[1] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms. MIT Press, 4th edition, 2022. (Chương 22: Elementary Graph Algorithms; Chương 24: Single-Source Shortest Paths).

[2] Edsger W. Dijkstra. A note on two problems in connexion with graphs. Numerische Mathematik, 1(1):269–271, 1959.

[3] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2):100–107, 1968.

[4] Robert Sedgewick and Kevin Wayne. Algorithms. Addison-Wesley Professional, 4th edition, 2011. (Chương 4: Graphs - Shortest Paths and Priority Queues).

[5] Daniel J. Bernstein. Comp.lang.c FAQ list · Question 17.1 (DJB2 Hash Function Algorithm), 1991.